Interfaces
Blynk
DMX Output
DMX Input
E1.31 (DMX) / Art-Net
HTTP Request API
Remote Control
JSON API
MQTT
Philips Hue
Serial
WLED UDP Sync
UDP Realtime / tpm2.net
Websocket
DDP Realtime

JSON API

WLED versions since 0.8.4 implement a powerful JSON API over HTTP.
It is accessible using the `/json` subpage.

Obtaining light information

Sending a GET request will return an object similar to the sample below The response consists of four objects:

- `state` contains the current state of the light. All values may be modified by the client (see below)
- `info` contains general information about the device. No value can be modified using this API
- `effects` contains an array of the effect mode names
- `palettes` contains an array of the palette names

You may also obtain those objects individually using the URLs `/json/state` `/json/info` `/json/eff`, and `/json/pal`.

Reserved effect IDs

In WLED versions 0.14+, some effects are unsupported in certain builds (e.g. some audio reactive effects may only work on ESP32). In order for each effect to have an unique ID on all devices, having unsupported ones in between supported ones is possible. If called, these will fallback to the Solid effect, in the effects list they have the name `RSVD` or `-`. To improve user experience, it is recommended to remove effects with the names `RSVD` or `-` form the UI effect selection.

Example Library

WLED JSON API Library in rust. Even if you are not using rust, or don't know how to read rust, the up-to-date JSON structure is included and documented in this project.

Setting new values

Sending a POST request to `/json` or `/json/state` with (parts of) the state object will update the respective values. Example: `{"on":true,"bri":255}` sets the brightness to maximum. `{"seg":[{"col":[[0,255,200]]}]}` sets the color of the first segment to teal. `{"seg":[{"id":X,"on":"t"}]}` and replacing X with the desired segment ID will toggle on or off that segment.

CURL example

This will toggle on and off and return the new state (v0.13+): `curl -X POST "http://[WLED-IP]/json/state" -d '{"on":"t","v":true}' -H "Content-Type: application/json"`

Sample JSON API response (v0.8.4):

```
{
  "state": {
    "on": true,
    "bri": 127,
    "transition": 7,
    "ps": -1,
    "pl": -1,
    "nl": {
      "on": false,
      "dur": 60,
      "fade": true,
      "tbri": 0
    },
    "udpn": {
      "send": false,
      "recv": true
    },
    "seg": [{
      "start": 0,
```

Table of contents
Obtaining light information
Example Library
Setting new values
Overview of values
State object
Contents of the segment object
Info object
Per-segment individual LED control
Playlists
Light capabilities
CCT control
Supported value ranges
Effect of the seg.cct value
Effect metadata
Effect parameters
Colors
Palette
Flags
Defaults
Sensors
Sensor object
Sensor types
API

```
      "stop": 20,
      "len": 20,
      "col": [
        [255, 160, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0]
      ],
      "fx": 0,
      "sx": 127,
      "ix": 127,
      "pal": 0,
      "sel": true,
      "rev": false,
      "cln": -1
    }]
  },
  "info": {
    "ver": "0.8.4",
    "vid": 1903252,
    "leds": {
      "count": 20,
      "rgbw": true,
      "pin": [2],
      "pwr": 0,
      "maxpwr": 65000,
      "maxseg": 1
    },
    "name": "WLED Light",
    "udpport": 21324,
    "live": false,
    "fxcount": 80,
    "palcount": 47,
    "arch": "esp8266",
    "core": "2_4_2",
    "freeheap": 13264,
    "uptime": 17985,
    "opt": 127,
    "brand": "WLED",
    "product": "DIY light",
    "btype": "src",
    "mac": "60019423b441"
  },
  "effects": [
    "Solid", "Blink", "Breathe", "Wipe", "Wipe Random", "Random Colors", "Sweep",
    "Scan", "Dual Scan", "Fade", "Chase", "Chase Rainbow", "Running", "Saw", "Twinl
    "Sparkle", "Dark Sparkle", "Sparkle+", "Strobe", "Strobe Rainbow", "Mega Strob
    "Chase Rainbow", "Chase Flash", "Chase Flash Rnd", "Rainbow Runner", "Colorful
    "Scanner", "Lighthouse", "Fireworks", "Rain", "Merry Christmas", "Fire Flicker
    "Out Out", "Out In", "Circus", "Halloween", "Tri Chase", "Tri Wipe", "Tri Fade
    "Dual Scanner", "Stream 2", "Oscillate", "Pride 2015", "Juggle", "Palette", "F
    "Noise 2", "Noise 3", "Noise 4", "Colortwinkle", "Lake", "Meteor", "Smooth Met
  ],
  "palettes": [
    "Default", "Random Cycle", "Primary Color", "Based on Primary", "Set Colors",
    "Forest", "Rainbow", "Rainbow Bands", "Sunset", "Rivendell", "Breeze", "Red & 
    "Pastel", "Sunset 2", "Beech", "Vintage", "Departure", "Landscape", "Beach", "
    "Drywet", "Jul", "Grintage", "Rewhi", "Tertiary", "Fire", "Icefire", "Cyane",
    "Magenta", "Magred", "Yelmag", "Yelblu", "Orange & Teal", "Tiamat", "April Nigl
  ]
}
```


Overview of values

State object

JSON key	Value range	Description
on	bool	On/Off state of the light. You can also use `"t"` instead of `true` or `false` to toggle.
bri	0 to 255	Brightness of the light. If *on* is `false`, contains last brightness when light was on (aka brightness when *on* is set to true). Setting *bri* to 0 is supported but it is recommended to use the range 1-255 and use `on: false` to turn off. The state response will never have the value `0` for *bri*.
transition	0 to 65535	Duration of the crossfade between different colors/brightness levels. One unit is 100ms, so a value of `4` results in atransition of 400ms.
tt	0 to 65535	Similar to transition, but applies to just the current API call. Not included in state response.
ps	-1 to 250	ID of currently set preset. `1~17~` can be used to iterate through presets 1-17, or `4~10r` to select random preset between presets 4 and 10 (inclusive).
~~pss~~	0 to 65535	Bitwise indication of preset slots (0 - vacant, 1 - written). Always 0 in 0.11. Not changable. *Removed as of v0.11.1*
psave	1 to 250 (16 prior to 0.11)	Save current light config (state) to specified preset slot. Not included in state response.
sb	bool	Used with `psave`. Save segment bounds (`start` & `stop`).
ib	bool	Used with `psave`. Save brightness.
sc	bool	Used with `psave`. Save selected segments.
pl	-1 to 250	ID of currently set playlist. *(read-olny)*
pdel	1 to 250	Preset ID to delete. Not included in state response.
nl.on	bool	Nightlight currently active
nl.dur	1 to 255	Duration of nightlight in minutes
~~nl.fade~~	bool	If `true`, the light will gradually dim over the course of the nightlight duration. If `false`, it will instantly turn to the target brightness once the duration has elapsed. *Removed in 0.13.0* (use mode instead)
nl.mode	0 to 3	Nightlight mode (0: instant, 1: fade, 2: color fade, 3: sunrise) (available since 0.10.2)
nl.tbri	0 to 255	Target brightness of nightlight feature
nl.rem	-1 to 15300	Remaining nightlight duration in seconds, -1 if not active. Only in state response, can not be set.
udpn.send	bool	Send WLED broadcast (UDP sync) packet on state change
udpn.recv	bool	Receive broadcast packets
udpn.sgrp	0 to 255	Bitfield for broadcast send groups 1-8
udpn.rgrp	0 to 255	Bitfield for broadcast receive groups 1-8
udpn.nn	bool	Don't send a broadcast packet (applies to just the current API call). Not included in state response.
v	bool	If set to *true* in a JSON POST command, the response will contain the full JSON state object. Not included in state response
rb	bool	If set to *true*, device will reboot immediately. Not included in state response.
live	bool	If set to *true*, enters realtime mode and blanks the LEDs. The realtime timeout option does not have an effect when this command is used, WLED will stay in realtime mode until the state (color/effect/segments, excluding brightness) is changed. It is expected that `{"live":false}` is sent once live data sending is terminated. Not included in state response.
lor	0, 1, or 2	Live data override. 0 is off, 1 is override until live data ends, 2 is override until ESP reboot (available since 0.10.0)
time	uint32	Set module time to unix timestamp. Not included in state response.

mainseg	0 to info.leds.maxseg-1	Main Segment
seg	Object or Array of objects	*(see below)* Segments are individual parts of the LED strip. Since 0.9.0 this enables running different effects on differentparts of the strip.
playlist	object	Custom preset playlists. Not included in state response. *(available since 0.11.0)*
tb	uint32	Sets timebase for effects. Not reported.
ledmap	0 to 9	Load specified ledmap (0 for `ledmap.json`, 1-9 for `ledmap1.json` to `ledmap9.json`). See mapping. Not included in state response. *(available since 0.14.0)*
rmcpal	bool	Remove last custom palette if set to `true`. Not included in state response. *(available since 0.14.0)*
np	bool	Advance to the next preset in a playlist if set to `true`. Not included in state response. *(available since 0.15)*

Contents of the segment object

Notice: *start*, *stop*, and *len* are not changeable in 0.8.4. Any segment with *id* > 0 is ignored. Unless stated otherwise, every value may be changed via an HTTP POST request. The tertiary color is not gamma-corrected in 0.8.4, but is in subsequent releases.

JSON key	Value range	Description
id	0 to info.maxseg -1	Zero-indexed ID of the segment. May be omitted, in that case the ID will be inferred from the order of the segment objects in the *seg* array.
start	0 to info.leds.count -1	LED the segment starts at. For 2D set-up it determines column where segment starts, from top-left corner of the matrix.
stop	0 to info.leds.count	LED the segment stops at, not included in range. If *stop* is set to a lower or equal value than *start* (setting to `0` is recommended), the segment is invalidated and deleted. For 2D set-up it determines column where segment stops, from top-left corner of the matrix.
startY	0 to matrix width	Start row from top-left corner of a matrix. *(available since 0.14.0)*
stopY	1 to matrix height	Stop row from top-left corner of matrix. *(available since 0.14.0)*
len	0 to info.leds.count	Length of the segment (*stop* - *start*). *stop* has preference, so if it is included, *len* is ignored.
grp	0 to 255	Grouping (how many consecutive LEDs of the same segment will be grouped to the same color)
spc	0 to 255	Spacing (how many LEDs are turned off and skipped between each group)
of	-len+1 to len	Offset (how many LEDs to rotate the virtual start of the segments, available since 0.13.0)
col	array of colors	Array that has up to 3 color arrays as elements, the primary, secondary (background) and tertiary colors of the segment. Each color is an array of 3 or 4 bytes, which represents a RGB(W) color, i.e. `[[255,170,0],[0,0,0],[64,64,64]]`. It can also be represented as aan array of strings of *hex* values, i.e. `["FFAA00","000000","404040"]` for orange, black and grey.
fx	0 to info.fxcount -1	ID of the effect or `~` to increment, `~-` to decrement, or `"r"` for random.
sx	0 to 255	Relative effect speed. `~` to increment, `~-` to decrement. `~10` to increment by 10, `~-10` to decrement by 10.
ix	0 to 255	Effect intensity. `~` to increment, `~-` to decrement. `~10` to increment by 10, `~-10` to decrement by 10.
c1	0 to 255	Effect custom slider 1. Custom sliders are hidden or displayed and labeled based on effect metadata.
c2	0 to 255	Effect custom slider 2.
c3	0 to 31	Effect custom slider 3.
o1	bool	Effect option 1. Custom options are hidden or displayed and labeled based on effect metadata.
o2	bool	Effect option 2.
o3	bool	Effect option 3.
pal	0 to info.palcount -1	ID of the color palette or ~ to increment, ~- to decrement, or r for random.
sel	bool	`true` if the segment is selected. Selected segments will have their state (color/FX) updated by APIs that don't support segments (e.g. UDP sync, HTTP API). If no segment is selected, the first segment (*id*: `0`) will behave as if selected. WLED will report the state of the first (lowest *id*) segment that is selected to APIs (HTTP, MQTT, Blynk...), or `mainseg` in case no segment is selected and for the UDP API. Live data is always applied to all LEDs regardless of segment configuration.
rev	bool	Flips the segment (in horizontal dimension for 2D set-up), causing animations to change direction.
rY	bool	Flips the 2D segment in vertical dimension. *(available since 0.14.0)*
on	bool	Turns on and off the individual segment. *(available since 0.10.0)*
bri	0 to 255	Sets the individual segment brightness *(available since 0.10.0)*
mi	bool	Mirrors the segment (in horizontal dimension for 2D set-up) *(available since 0.10.2)*
mY	bool	Mirrors the 2D segment in vertical dimension. *(available since 0.14.0)*

tp	bool	Transposes a segment (swaps X and Y dimensions). *(available since 0.14.0)*
cct	0 to 255 *or* 1900 to 10091	White spectrum color temperature *(available since 0.13.0)*
lx	`BBBGGGRRR` : 0 - 100100100	Loxone RGB value for primary color. Each color (`RRR` , `GGG` , `BBB`) is specified in the range from 0 to 100%. *Only available if Loxone is compiled in.*
lx	`20bbbtttt` : 200002700 - 201006500	Loxone brightness and color temperature values for primary color. Brightness `bbb` is specified in the range 0 to 100%. `tttt` defines the color temperature in the range from 2700 to 6500 Kelvin. (available since 0.11.0, not included in state response) *Only available if Loxone is compiled in.*
ly	`BBBGGGRRR` : 0 - 100100100	Loxone RGB value for secondary color. Each color (`RRR` , `GGG` , `BBB`) is specified in the range from 0 to 100%. *Only available if Loxone is compiled in.*
ly	`20bbbtttt` : 200002700 - 201006500	Loxone brightness and color temperature values for secondary color. Brightness `bbb` is specified in the range 0 to 100%. `tttt` defines the color temperature in the range from 2700 to 6500 Kelvin. *(available since 0.11.0, not included in state response) Only available if Loxone is compiled in.*
i	array	Individual LED control. Not included in state response *(available since 0.10.2)*
frz	bool	freezes/unfreezes the current effect
m12	0 to 4 [map1D2D.count]	Setting of segment field 'Expand 1D FX'. (0: Pixels, 1: Bar, 2: Arc, 3: Corner)
si	0 to 3	Setting of the sound simulation type for audio enhanced effects. (0: 'BeatSin', 1: 'WeWillRockYou', 2: '10_3', 3: '14_3') (*as of 0.14.0-b1, there are these 4 types defined*)
fxdef	bool	Forces loading of effect defaults (speed, intensity, etc) from effect metadata. *(available since 0.14.0)*
set	0 to 3	Assigns group or set ID to segment (not to be confused with *grouping*). Visual aid only (helps in UI). *(available since 0.14.0)*
rpt	bool	Flag to repeat current segment settings by creating segments until all available LEDs are included in automatically created segments or maximum segments reached. Will also toggle *reverse* on every *even* segment. *(available since 0.13.0)*

Info object

No value may be changed by means of this API.

JSON key	Value range	Description
ver	string	Version name.
vid	uint32	Build ID (YYMMDDB, B = daily build index).
leds	object	Contains info about the LED setup.
leds.cct	bool	`true` if the light supports color temperature control *(available since 0.13.0, deprecated, use info.leds.lc)*
leds.count	1 to 1200	Total LED count.
leds.fps	0 to 255	Current frames per second. *(available since 0.12.0)*
leds.rgbw	bool	`true` if LEDs are 4-channel (RGB + White). *(deprecated, use info.leds.lc)*
leds.wv	bool	`true` if a white channel slider should be displayed. *(available since 0.10.0, deprecated, use info.leds.lc)*
~~leds.pin~~	byte array	LED strip pin(s). Always one element. *Removed as of v0.13*
leds.pwr	0 to 65000	Current LED power usage in milliamps as determined by the ABL. `0` if ABL is disabled.
leds.maxpwr	0 to 65000	Maximum power budget in milliamps for the ABL. `0` if ABL is disabled.
leds.maxseg	byte	Maximum number of segments supported by this version.
leds.lc	byte	Logical AND of all active segment's virtual light capabilities
leds.seglc	byte array	Per-segment virtual light capabilities
str	bool	If `true`, an UI with only a single button for toggling sync should toggle receive+send, otherwise send only
name	string	Friendly name of the light. Intended for display in lists and titles.
udpport	uint16	The UDP port for realtime packets and WLED broadcast.
live	bool	If `true`, the software is currently receiving realtime data via UDP or E1.31.
lm	string	Info about the realtime data source
lip	string	Realtime data source IP address
ws	-1 to 8	Number of currently connected WebSockets clients. -1 indicates that WS is unsupported in this build.
fxcount	byte	Number of effects included.
palcount	uint16	Number of palettes configured.
wifi	object	Info about current signal strength
wifi.bssid	string	The BSSID of the currently connected network.
wifi.signal	0 to 100	Relative signal quality of the current connection.
wifi.channel	1 to 14	The current WiFi channel.
fs	object	Info about the embedded LittleFS filesystem (since 0.11.0)
fs.u	uint32	Estimate of used filesystem space in kilobytes
fs.t	uint32	Total filesystem size in kilobytes
fs.pmt	uint32	Unix timestamp for the last modification to the `presets.json` file. Not accurate after boot or after using `/edit`
ndc	-1 to 255	Number of other WLED devices discovered on the network. -1 if Node discovery disabled. (since 0.12.0)
arch	string	Name of the platform.
core	string	Version of the underlying (Arduino core) SDK.
lwip	0, 1, or 2	Version of LwIP. 1 or 2 on ESP8266, 0 (does not apply) on ESP32. *Deprecated, removal in 0.14.0*

freeheap	uint32	Bytes of heap memory (RAM) currently available. Problematic if < `10k`.
uptime	uint32	Time since the last boot/reset in seconds.
opt	uint16	Used for debugging purposes only.
brand	string	The producer/vendor of the light. Always `WLED` for standard installations.
product	string	The product name. Always `FOSS` for standard installations.
~~btype~~	string	The origin of the build. `src` if a release version is compiled from source, `bin` for an official release image, `dev` for a development build (regardless of src/bin origin) and `exp` for experimental versions. `ogn` if the image is flashed to hardware by the vendor. *Removed as of v0.10*
mac	string	The hexadecimal hardware MAC address of the light, lowercase and without colons.
ip	string	The IP address of this instance. Empty string if not connected. (since 0.13.0)

Examples of frequently requested custom API:

Function/Effect	API (Add to preset or call from other sources)
Cycle presets between 1 and 6	`{"ps":"1~6~"}`
Select random effect on *all selected* segments	`{"seg":{"fx":"r"}}`
Select random palette between 5 and 10 on segment 2	`{"seg":[{"id":2,"pal":"5~10r"}]}`
Change segment 0 name	`{"seg":[{"id":0,"n":"Your custom ASCII text"}]}`
Freeze or unfreeze an effect	`{"seg":[{"id":0,"frz":true}]}` or `{"seg":[{"id":0,"frz":false}]}`
Night light	`{"nl":{"on":true,"dur":10,"mode":0}}`
Increase brightness by 40 wrapping when maximum reached	`{"bri":"w~40"}`

Per-segment individual LED control

Using the `i` property of the segment object, you can set the LED colors in the segment using the JSON API.
Keep in mind that this is non-persistent, if the light is turned off the segment will return to effect mode.
The segment is frozen when using individual control, the set effect will not run.
To unfreeze the segment, click the "eye" icon, change any property of the segment or turn off the light.

To set individual LEDs starting from the beginning, use an array of Color arrays `[255,0,0]` or hex values `"FF0000"`. Hex values are more efficient than Color arrays and should be preferred when setting a large number of colors.
`{"seg":{"i":["FF0000","00FF00","0000FF"]}}` or `{"seg":{"i":[[255,0,0],[0,255,0],[0,0,255]]}}` will set the first LED red, the second green and the third blue.

To set individual LEDs, use the LED index followed by its color value.
`{"seg":{"i":[0,"FF0000",2,"00FF00",4,"0000FF"]}}` is the same as above, but leaves blank spaces between the lit LEDs.

To set ranges of LEDs, use the LED start and stop index followed by its color value.
`{"seg":{"i":[0,8,"FF0000",10,18,"0000FF"]}}` sets the first eight LEDs to red, leaves out two, and sets another 8 to blue.

To set a large number of colors, send multiple api calls of 256 colors at a time.
`{"seg": {"i":[0,"CC0000","00CC00","0000CC","CC0000"...]}}` `{"seg": {"i":[256,"CC0000","00CC00","0000CC","CC0000"...]}}` `{"seg": {"i":[512,"CC0000","00CC00","0000CC","CC0000"...]}}`

Do not make several calls in parallel, that is not optimal for the device. Instead make your call in sequence, where each call waits for the previous to complete before making a new one. How this is done depends on your choice of tool, but with CURL you que your commands by separating then with `&&` i.e. `CURL [command 1] && CURL [command 2] && CURL [command 3]`.

Command buffer size

If you are trying to set many LEDs and it fails to work, you can check your request here for length. Select ESP32 and Deserialize. If the required buffer size is above 10K for ESP8266 and 24K for ESP32, please split it into multiple sequential requests and consider using the Hex string syntax.

Keep in mind that the LED indices are segment-based, so LED 0 is the first LED of the segment, not of the entire strip. Segment features, including Grouping, Spacing, Mirroring and Reverse are functional.

Matrices are handled as a non-serpentine layout.

Brightness interaction

For your colors to apply correctly, make sure the desired brightness is set beforehand. Turning on the LEDs from an off state and setting individual LEDs in the same JSON request will *not* work!

Playlists

(Available since 0.11.0)

Sample playlist API call:

```
{
  "playlist": {
    "ps": [26, 20, 18, 20],
    "dur": [30, 20, 10, 50],
    "transition": 0,
    "repeat": 10,
    "end": 21
  }
}
```

This example applies preset ID 26 for 3 seconds, then preset 20 for 2 seconds, then preset 18 for 1 second, lastly preset 20 again for 5 seconds.This repeats 10 times, then preset 21 is applied.

Playlist object:

JSON key	Description
ps	Array of preset ID integers to be applied in this order.
dur	Array of time each preset should be kept, in tenths of seconds. If only one integer is supplied, all presets will be kept for that time.Defaults to 10 seconds if not provided.
transition	Array of time each preset should transition to the next one, in tenths of seconds. If only one integer is supplied, all presets will transition for that time. Defaults to the current transition time if not provided.
repeat	How many times the entire playlist should cycle before finishing. Set to `0` for an indefinite cycle. Default to indefinite if not provided.
end	Single preset ID to apply after the playlist finished. Has no effect when an indefinite cycle is set. If not provided, the light will stay on the last preset of the playlist.

Light capabilities

In order to e.g. only show color controls relevant to a given setup, it is necessary to obtain the color capabilities of the light.
The `info.leds.seglc` array can be used to do so on a per-segment level. It contains `n+1` 8-bit integers, where `n` is the `id` of the last *active* segment, each index corresponds to the segment with that ID.
This integer indicates whether a given segment supports (24 bit) RGB colors, an extra (8 bit) white channel and/or adjustable color temperature (CCT):

Bit	Capability
0	Segment supports RGB color
1	Segment supports white channel
2	Segment supports color temperature
3-7	Reserved (expect any value)

Therefore:

`lc` value	Capabilities
0	None. Indicates a segment that does not have a bus within its range, e.g. because it is not active.
1	Supports RGB
2	Supports white channel only
3	Supports RGBW
4	Supports CCT only, no white channel (unused)
5	Supports CCT + RGB, no white channel (unused)
6	Supports CCT (including white channel)
7	Supports CCT (including white channel) + RGB

Note that CCT is controllable per-segment, while RGB color and white channel have 3 color slots each per segment.

`info.leds.lc` contains this info on a global level, and is a bitwise AND of the per-segment light capability values.

CCT control

Please also see the general info about CCT.

SUPPORTED VALUE RANGES

Given that the white spectrum handling is agnostic to the true color temperature of the LEDs used, a relative range is preferred for the time being, where a value of `0` indicates the warmest possible color temperature, while a value of `255` indicates the coldest temperature.

It is also possible to pass a value in the range of `1900` to `10091`, in which case it is treated as a Kelvin color temperature, where `1900` is mapped to a relative value of `0` and `10091` to a relative value of `255`.

As such, it is unlikely to match the *actual* color temperature output by the light, therefore the relative values 0-255 are preferred for the time being.

In the future, an option to specify the Kelvin temperatures of the utilized hardware may be added, once this is done, a color temperature can be set to more accurately match other lights.

Therefore, for forward compatibility, your integration should expect both either a 0-255 value for `seg.cct`, in which case it is a relative value, or an absolute Kelvin value in the range 1000-16000 K. In case a Kelvin value is provided, you can consider the color temperature as accurate, which is not possible with relative 0-255 values as the Kelvin points of the white channels are unknown. It is preferred that you set a new CCT value in the same range as received from WLED, that is, use 0-255 if the original value was within this range, and 1000-20000 K otherwise.

If your code relies on absolute Kelvin values, a reasonable estimate for the warm white point (relative `0`) could be 2700K, while cold white (relative `255`) could commonly be 6500K.

EFFECT OF THE SEG.CCT VALUE

`seg.cct` can always be set, but only has an effect on the physical state of the light if one or both of the following conditions is met:

- White Balance correction is enabled
- A bus supporting CCT is configured and `Calculate CCT from RGB` is *not* enabled

CCT support is indicated by `info.leds.cct` being `true`, in which case you can regard the instance as a CCT light and e.g. display a color temperature control.

Effect metadata

> **Why effect metadata?**
>
> Prior to 0.14, user interfaces showed Speed and Intensity slider, palette controls, and all three color slots regardless of the effect selected. This may cause confusion to the user because controls are displayed that have no immediate effect in the current configuration. Effect metadata allows you to dynamically hide certain controls, so that the user only sees controls actually utilized by the selected effect mode.

Starting with WLED 0.14, effect metadata is available under the `/json/fxdata` URL.
This returns an array of strings with `info.fxcount` entries. The string at a given index corresponds to the metadata of the effect with the same ID as that index. Metadata is stored in a

memory-optimized string format, for example the Aurora effect has the metadata `!,!;;!;1;sx=24,pal=50`.

The metadata string consists of up to five sections, separated by semicolons: `<Effect parameters>;<Colors>;<Palette>;<Flags>;<Defaults>`

EFFECT PARAMETERS

The first section specifies the number and labels of effect parameters (e.g. speed, intensity). Up to 5 sliders and 3 checkboxes are supported (`sx`, `ix`, `c1`, `c2`, `c3`, `o1`, `o2`, `o3` parameters in the `seg` object). For more details about the ranges of the sliders see contents-of-the-segment-object. Slider/checkbox labels are comma separated. An empty or missing label disables this control. `!` specifies the default label is used:

Parameter	Default tooltip label
sx	Effect speed
ix	Effect intensity
c1	Custom 1
c2	Custom 2
c3	Custom 3
o1	Option 1
o2	Option 2
o3	Option 3

The fallback value if this section is missing is two sliders, Effect speed and Effect intensity.

Examples:

Parameter string	Displayed controls
`<empty>`	No effect parameters
!	1 slider: Effect speed
!,!	2 sliders: Effect speed + Effect intensity
!,Phase	2 sliders: Effect speed + Phase
,Saturation,,,,Invert	1 slider (sets `ix` parameter) and 1 checkbox: Saturation + Invert
,,,,,Random colors	1 checkbox: Random colors

COLORS

Up to 3 colors can be used. Please note that only the first two characters of the label are visible in the WLED UI.
`!` specifies the default label is used. The default labels for the color slots are `Fx`, `Bg`, and `Cs`.

The fallback value if this section is missing is 3 colors: `Fx` + `Bg` + `Cs`.

Examples:

Colors string	Displayed controls
`<empty>`	No color controls
!	1 color: Fx
,!	1 color: Bg
!,!	2 colors: Fx + Bg
1,2,3	3 colors: 1 + 2 + 3

PALETTE

If empty, the effect does not use palettes. If `!`, palette selection is enabled.

The fallback value if this section is missing is palette selection enabled.

FLAGS

Flags allow filtering for effects with certain characteristics. They are a single character each and not comma-separated. Currently, the following flags are specified:

Flag	Effect characteristic
0	Effect works well on a single LED. If flag 0 is present, flags 1/2/3 are omitted. (unused)
1	Effect is optimized for use on 1D LED strips.
2	Effect requires a 2D matrix setup (unless flag 1 is also present)
3	Effect requires a 3D cube (unless flags 1 and/or 2 are also present) (unused)
v	Effect is audio reactive, reacts to amplitude/volume.
f	Effect is audio reactive, reacts to audio frequency distribution.

For example, a Flag string of `2v` denotes a volume reactive effect that is to be used on 2D matrices.

The fallback value if this section is missing is `1`, i.e. a 1D optimized effect.

DEFAULTS

Defaults are values for effect parameters that work particularly well on that effect. They are set automatically when the effect is selected in UI unless configured otherwis in UI settings. To specify defaults, use the standard segment parameter name (e.g. `ix`) followed by an `=` and the default value. For example, `sx=24,pal=50` sets the effect speed to 24 (slow) and the palette to ID 50 (Aurora).

If no default is specified for a given parameter, it retains the current value.

Sensors

> **Warning**
>
> This section about the Sensor API is a DRAFT specification. It is not yet implemented and subject to change.

Various types of sensors (e.g. for Temperature, light intensity, PIR) may be added to WLED via usermods. To allow read access to sensor data via the JSON API in a standardized way, the `info.sensor` array is used.

If the `info.sensor` array is missing or empty, no sensor values are exposed.

Sensor object

Each sensor/measurement is represented by an object within the `info.sensor` array.

For example,

```
{"type":"T","n":"Outside","val":12}
```

refers to a 12 °C temperature measurement in Celsius with the sensor name "Outside".

The object may contain the following properties, of which all are optional, except `type`.

JSON key	Value range	Description
type	string	The type of the sensor.
n	string	The name of the sensor. If omitted, the client may generate a suitable name (e.g. "Temperature sensor 1")
val	any	The most current sensor reading. May be of any JSON type depending on the type of the sensor, this is a number for all sensor types pre-defined below except for the `"b"` and `"CL"` types and custom type sensors. `null` if the reading is invalid, either due to an error or because the first reading has not yet completed.
unit	string	An explicit human-readable unit string for the measurement. If omitted, the default for the sensor type is used.
error	int or string	If present and not `null`, `false`, `0` or an empty string, a sensor error is indicated. May either be an integer error code or an error string.
tc	number	Seconds of WLED `uptime` when the value last changed substantially. The threshold for a "substantial" change is up to the implementation. This can for example be used to find when a PIR sensor was last activated.
tm	number	Seconds of WLED `uptime` when the last reading given by `val` was obtained.
ts	number	Seconds of WLED `uptime` at the first measurement / start of measurement period. (required for Energy sensor type)
min	number	Lower bound of possible value range
max	number	Upper bound of possible value range
u	number	Absolute uncertainty of the measurement
model	string	Identification of the sensor hardware used

Sensor types

These are the standardized sensor types that may be implemented by usermods:

Type ID string	Measurement type	Default unit
"" (empty string)	Invalid sensor (reserved)	-
b	Button/Boolean	true/false
c	Custom user-defined sensor	-
q	Electric charge	As
t	Time	s
BL	Battery Level	%
CL	24-bit RGB color	hex string
E	Energy (`ts` property required)	J
I	Electric current	A
J	Illuminance	lx
L	Distance	m
Lp	Sound pressure level	dB
M	Mass	kg
N	Number/count	-
P	Power	W
Pe	General purpose percentage	%
PL	Power Level (signal strength)	dBm
Pr	Pressure	Pa
R	Electric Resistance	Ohms
RH	Relative Humidity	%
T	Temperature	°C
U	Voltage	V
(other strings)	Reserved, let us know if you need a new type added	-

If a client is only interested in certain sensor types (e.g. Temperature), it may disregard all other sensor objects.

API

there is all routes for JSON API:

- /json/state
- /json/info
- /json/si
- /json/nodes
- /json/eff
- /json/palx
- /json/fxdata
- /json/net
- /json/live // only if flag WLED_ENABLE_JSONLIVE is on
- /json/pal
- /json/cfg